

appropriate kernel subsystem

- Clean up process resources (final stage of the process)

There are two types of interfaces for this — a complete interface for the kernel system and a limited one for user processes. A process can initiate other processes by copying the existing process. For example, when the system is booting, only *init* will be running. Then the *fork()* system call is used to spawn off copies. This means that it creates a new child that is a true copy of its parent.

You can see that the process scheduler is also vital for loading, execution, and the proper termination of the user processes.

The structure *task\_struct* is used to refer a task. You can find a field that is used to indicate the state. That may have any of the following states: ready, waiting, running, returning from a system call, processing the INT routine and processing SC. You can also find fields that carry information about the clock interval and priority. From this, process ID information can be retrieved. If you take a look at the *files\_struct* (which is a substructure), you can see the list of files opened by the process. Fields concerning the amount of time the process has spent, can also be located.

Now we can discuss the aspects concerning memory management. Here are a few of the main points concerning the unique features:

- A large pool of address space (so that user programs can refer more memory than the physically available one).
- Memory for a process is private and it cannot be modified by another process. The memory manager restricts processes from overwriting code and any read-only data.
- The memory-mapping feature can map a file into a portion of virtual memory and access the file as memory.
- The *Fair Access to Physical Memory* feature offers good system performance.
- The memory manager allows processes to share portions of their memory.

The memory manager offers two interfaces—a system-call interface that's used by user processes and another interface used by the kernel subsystems to perform their actions. Please see the sidebar titled 'System-call interface' for a detailed review.

## Filesystem

We have already seen that Linux has been ported to various platforms ranging from computers to wristwatches. We know that even for one particular device, say a hard drive, there are many differences in the interfaces used by different vendors. Linux supports a large number of logical filesystems. Thus,

## SYSTEM-CALL INTERFACE

### System-call interface

- *mprotect()*: To change the protection on the virtual memory portion
- *mmap()/munmap()/msync()/mremap()*: To map files into the virtual memory portions
- *mlock()/mlockall()/munlock()/munlockall()*: Super-user routines to refrain memory from being swapped
- *swapon()/swaponoff()*: Super-user routines to add and remove swap files
- *malloc()/free()*: To allocate or free a portion of the memory for the use of a given process

### Intra-kernel interface

- *verify\_area()*: To verify that a portion of the user memory is mapped with the necessary permissions
- *kmalloc()/kfree()*: To allocate and free memory for use by the data structures of the kernel
- *get\_free\_page()/free\_page()*: To allocate and free memory pages

inter-operations are made possible. The filesystem has the following advantages:

- Supports multiple hardware devices
- Supports multiple logical filesystems
- Supports multiple executable formats
- Offers a common interface to the logical filesystems
- Provides high-speed access to files
- Can restrict a user's access to files and the user's total file size, with quotas

There are two levels of interfaces here—a system-call interface for the user processes and an internal interface for other kernel subsystems. File subsystems expose the data structures and the implementation function for the direct manipulation by other kernel subsystems. You may note that two interfaces are exposed, viz., inodes and files. Please glance at the box for more information.

We have reached the end of today's voyage. I look forward to your feedback so that I can incorporate your ideas into our next voyage.

Happy kernel hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at [www.aasisvinayak.com](http://www.aasisvinayak.com)